

МОЯ КАРТА СИСТЕМИ / БЕЗ МАРКЕТИНГОВИХ ОБІЦЯНОК

Що насправді знаходиться всередині e-osvita

Розкладаю проєкт по частинах: що бачить студент, що робить worker, де лежать дані та які місця поки не можна називати завершеними.

Backend + Worker + Admin SPA

Node.js / React / PostgreSQL

Оновлено: 05.09.2026

WORKING COPY

1. Загальна інформація

e-osvita — серверна освітня платформа для мобільного застосунку студента та внутрішньої адміністративної панелі. Вона об'єднує облікові записи користувачів, структуру університету, навчальні дані, фінансову інформацію та фонову синхронізацію із зовнішньою інформаційною системою закладу освіти.

Основна ідея: не звертатися до системи університету під час кожної дії студента, а періодично отримувати дані, перевіряти їх і зберігати локальну актуальну копію. Це прискорює мобільний API та дозволяє продовжувати читання вже синхронізованих даних під час короткочасної недоступності зовнішнього сервісу.

Продукт

Мобільний API для студентів і web-панель для операторів.

Предметна область

Освіта, розклад, оцінки, навчальний план, оплата та новини.

Джерела

PostgreSQL, API університету, банківський процесинг, OTP/SMS/email.

Фонові процеси

BullMQ worker із Redis, повторні спроби та контроль станів.

2. Користувачі системи

Користувач	Потреба	Можливості
Студент	Мати навчальні сервіси в одному застосунку	Профіль, розклад, екзамени, оцінки, план, оплата, знижки, новини
Оператор університету	Перевіряти дані та вирішувати проблеми	Пошук студента, інспектор, ручна синхронізація, перегляд помилок
Адміністратор платформи	Керувати всією системою	Організації, користувачі, jobs, cron, failover, очищення даних
ІТ-служба університету	Підтримувати обмін	Provider API, тестове середовище, моніторинг доступності
Служба підтримки	Розслідувати звернення	Пошук, статуси, дати оновлення та технічні помилки

У базовій моделі існують ролі `GUEST`, `STUDENT`, `TEACHER` та `ADMIN`. Поточні адміністративні права глобальні; для повноцінної роботи багатьох університетів потрібні ролі та доступи на рівні організації.

3. Функціональні можливості

Реєстрація та доступ Підтвердження телефону через OTP, пароль, login, access/refresh JWT, відновлення доступу, зміна телефону та email.
Верифікація студента Пошук людини у зовнішній системі за телефоном та створення локального профілю студента.
Навчальні дані Курс, семестр, форма навчання, спеціальність, групи, дисципліни, оцінки й середній бал.

Розклад

Лекції, заняття, лабораторні, семінари, консультації, онлайн-події та екзамени.

Навчальний календар

Періоди навчання, сесії, канікули та інші елементи плану.

Фінанси студента

Стан оплати навчання, історія зарахувань, знижки та пільги.

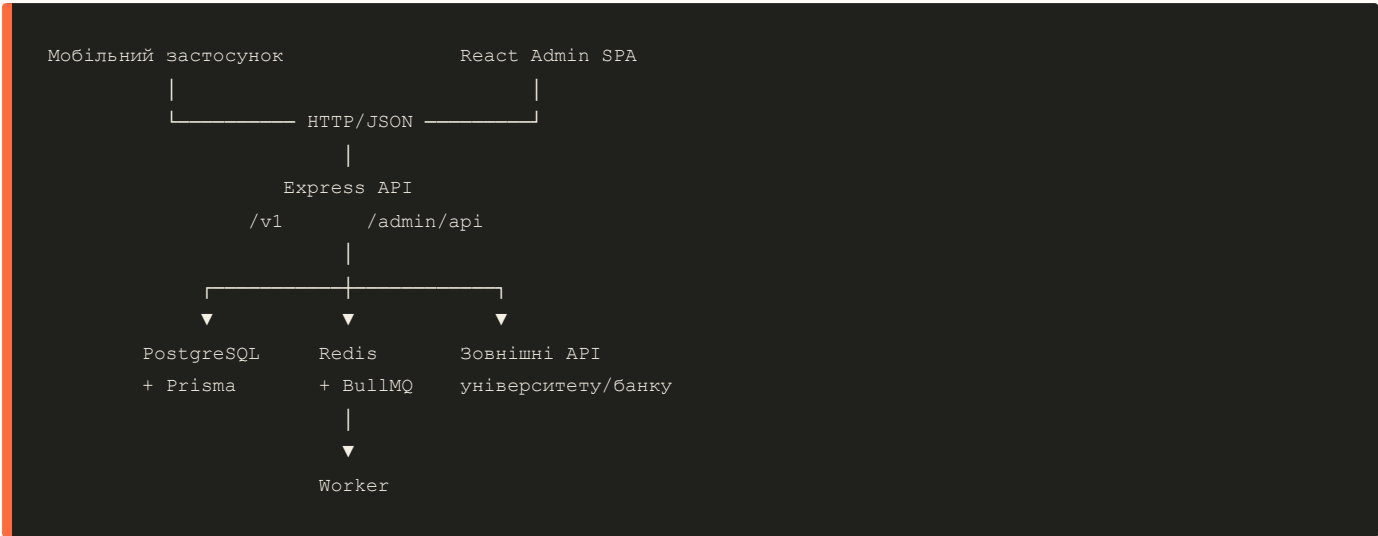
Платежі

Платіжні шаблони, payment link, callback/webhook, 3-D Secure та токенизація карток.

Новини

Отримання RSS-публікацій та прив'язка матеріалів до університету.

4. Компоненти платформи



Компонент	Технології	Відповідальність
Backend API	Node.js, TypeScript, Express	Маршрути, авторизація, validation і бізнес-операції
Worker	BullMQ	Виконання тривалих та періодичних синхронізацій
Database	PostgreSQL, Prisma	Користувачі, навчальні, фінансові й операційні дані
Queue	Redis	Черги, повторні спроби, cron-завдання
Admin SPA	React, Vite, TanStack Query, Zustand	Моніторинг і робота операторів
Observability	Winston, Morgan, Kafka/Logstash	HTTP-логи та передача подій у зовнішню систему логування

5. Модель даних

```
Organization
├─ Department - Cafedra - Lecturer
│   └─ UserStudent - StudentGroup - Group
│       └─ Discipline
├─ Building - Auditorium - Schedule
├─ OrganizationArticle
├─ PaymentTemplate
└─ OrganizationSyncStatus

User
├─ UserStudent
│   ├── StudentSchedules
│   ├── Score / AverageScore
│   ├── EducationPlan
│   ├── PaymentHistory / Discounts
│   └─ StudentSyncStatus
├─ Token / PhoneVerification
├─ StudentPayment - PaymentTranState
└─ CardToken
```

Схема містить 28 моделей і 20 enum. Центральна зв'язка — `User → UserStudent → Department → Organization`. Один користувач може мати кілька студентських профілів. Синхронізація контролюється окремо для студента або організації та конкретного типу даних.

6. Основні процеси

Реєстрація та верифікація

```
Телефон → OTP → реєстрація User → JWT
    → вибір університету → /studentinfo
    → створення UserStudent → запуск синхронізацій
```

Періодичне оновлення

```
Cron → BullMQ → Worker → API університету
    → нормалізація → транзакції Prisma
    → оновлення SyncStatus → дані доступні застосунку
```

Оплата

PaymentTemplate → StudentPayment → банківська сесія
→ hosted payment або token purchase → 3DS
→ callback/webhook → PaymentTranState → фінальний статус

7. Адміністративна панель

React-панель містить dashboard, список студентів, інспектор студента, запрошення, керування jobs, список організацій та сторінку організації.

Розділ	Що дає	Поточне обмеження
Dashboard	Кількість станів, помилки, circuit breaker, організації	Не зберігається повна історія запусків
Students	Пошук, фільтри, перехід до інспектора	Ієрархія університет → кафедра → група обмежена
Inspector	Розклад, оцінки, оплати, знижки, план, sync status	Є небезпечні операції видалення; потрібен audit log
Jobs	Запуск, bulk trigger, cron, статистика	Потрібне посилення authorization і пояснення статусів
Organizations	Список, редагування, стан синхронізації	Немає branding, tenant-admins та integration settings
Payments	Частково видно оплату навчання в інспекторі	Окремого моніторингу банківських платежів немає

8. Зовнішні інтеграції

- **Система університету:** профіль, баланс, розклад, оцінки, календар, платежі, знижки та RSS.
- **OTP і SMS:** підтвердження телефону та відновлення доступу.
- **Email:** підтвердження адреси й сервісні повідомлення.
- **TASLink:** payment link, токенизація, покупка, 3-D Secure та статус транзакції.
- **Kafka/Logstash:** необов'язкова централізована доставка логів.

Поточна конфігурація університетського API глобальна. Для моделі «один університет — одна екосистема» потрібні окремі credentials, endpoint, branding, feature flags і графік синхронізації для кожного `organizationId`.

9. Безпека

Застосунок використовує JWT, Passport, password hashing, Joi validation, Helmet, XSS-фільтрацію, rate limiting, CORS і централізований error handler.

Основні ризики, які потребують закриття

- Нерівномірне застосування authorization до адміністративних endpoints.
- Глобальна роль ADMIN без обмеження університетом.
- Вимкнена перевірка TLS-сертифіката MAUP.
- Відсутність повного журналу адміністративних дій.
- Необхідність обмежити CORS конкретними origin.
- Фінансові та персональні дані потребують маскування в логах.
- Відомі npm vulnerabilities мають бути проаналізовані без автоматичного force-upgrade.

10. Розгортання та експлуатація

API та worker масштабуються окремо. PostgreSQL є основним сховищем, Redis — інфраструктурою черг. Для Kubernetes у репозиторії є Helm chart. Dockerfile збирає backend і React SPA.

Процес	Команда/артефакт	Примітка
Development API	<code>npm run dev</code>	Watch mode, порт 3000
Worker	<code>npm run worker:dev</code>	Повинен працювати окремо
Admin SPA	<code>admin-spa: npm run dev</code>	Vite, порт 5173
Міграції	<code>npm run db:migrate</code>	Prisma migrate deploy
Build	<code>npm run build</code>	TypeScript → build
Health	<code>/v1/actuator/health</code>	Потрібно додати readiness PostgreSQL/Redis

11. Поточні обмеження

- Архітектура ще не забезпечує повну tenant-ізоляцію кількох університетів.
- Немає моделей branding та налаштувань університету.
- Немає окремого UI для платіжних транзакцій і карткових токенів.
- Історія синхронізацій моделюється недостатньо: переважно зберігається поточний стан.

- Стара EJS-адмінка співіснує з React SPA.
- Маршрути ``/admin``, ``/admin/api`` та ``/api/admin`` налаштовані неузгоджено.
- Між Prisma-схемою та міграціями було виявлено розбіжність enum ``SUSPENDED``.
- Штатний test script посилається на відсутню команду ``db:push``.
- Назви деяких полів містять історичні помилки: ``semestr``, ``discaunt``, ``edutaion``, ``autidotirum``.

12. Рекомендований розвиток

1. Закрити адміністративні endpoints єдиною authentication та permission policy.
2. Додати tenant-моделі, полі університету та автоматичний organization scope.
3. Додати branding, feature flags і конфігурацію інтеграцій на університет.
4. Виправити міграції та перевіряти чисте розгортання БД у CI.
5. Додати історію запусків, correlation ID та audit log.
6. Стандартизувати Provider API через затверджений OpenAPI і runtime validation.
7. Додати платіжний operational dashboard.
8. Прибрати старі EJS-сторінки після перенесення функцій.
9. Уніфікувати маршрути frontend/backend/ingress.
10. Додати contract, security, isolation, migration і end-to-end tests.

Підсумок. e-osvita вже має основу повноцінної освітньої платформи: модель студента, навчальні дані, платежі, черги, worker та адміністративний інтерфейс. Основний наступний крок — перетворити одноінтеграційну систему на безпечну multi-tenant-платформу з контрольованим контрактом і доказовою ізоляцією університетів.